are binary safe; the few exceptions will be noted in this section. For now, here's a quick example:

```
var_dump(strlen("A\x00 null byte."));
var_dump(strcoll("abc", "abc\x00def"));
var_dump(strcmp("abc", "abc\x00def"));
```

```
Output:
int(13)
int(0)
int(-1)
```

The code above uses three different string functions. `strlen()` returns the length of the string (from the buffer length metadata); the native C equivalent would have returned `1` (terminating after the "A" byte), but PHP uses the buffer length metadata and returns `13`. `strcoll()` compares strings by locale (the set of rules for formatting and comparing data by region, we'll get to this shortly) and returns `0` if they match, which is the case here despite the two strings being obviously different (but matching up to the null byte of the second string). Clearly, `strcoll()` is not binary safe because the second argument contains seven bytes, not three; what's happened here is that evaluation of the second string terminated early at the null byte. `strcmp()` is also a very similar string comparison function to `strcoll()`, but *is* binary safe, so it returns `-1` (the left-hand argument is smallest, at three bytes).

Multi-byte Safety

In a similar vein to binary safety, multi-byte safety is a property of functions that are aware of multi-byte encoding systems and will differentiate between characters and bytes. PHP core offers many built-in functions for handling string data, but they do not differentiate between single-byte and multi-byte encodings and will give incorrect results when the text encoded in the string steps outside the ASCII range.

Here's a simple example: the function `strlen()` will return the length of a string. Let's look at a simple ASCII string vs. something outside the ASCII range… I'm choosing the Polish alphabet for this. Unless stated otherwise, all non-ASCII strings in this section are using PHP's default encoding of UTF-8.